

Design and Implementation of the "Gold Rim Shop" E-Commerce System

Module Title: Object-Oriented System Design Module Code: SEN5000
Assessment ID: PORT1
Academic Year: 2025/26

Author Name : Rim Zino

Author ID: st20320120

TABLE OF CONTENTS

SYSTEM DESIGN: USE CASE MODELING: 6
SYSTEM ARCHITECTURE: CLASS DIAGRAM & OOP PRINCIPLES: 7
DATABASE DESIGN: ENTITY-RELATIONSHIP MODELING: 9
THE LOGIN SCREEN: 11
User Interface Implementation: 11
THE REGISTRATION FORM: 12
User Registration Module: 12
THE PRODUCT CATALOG : 13
Product Catalog & Company Information: 13
THE SHOPPING CART: 14
Session-Based Shopping Cart: 14
CHECKOUT & PAYMENT SIMULATION: 15
Checkout and Payment Validation: 15
THE ORDER TRACKER: 16
Logic-Driven Order Tracking: 16
THE ADMIN DASHBOARD: 17
Administrator Dashboard: 17
Inventory & Order Management Modules: 18
THE SUPPORT CHAT: 19
Real-Time Customer Support System: 19
THE USERS TABLE: 21
User Data Persistence: 21
THE `ORDERS` TABLE: 21
Transactional Data Integrity: 21
THE `SUPPORTMESSAGE` TABLE: 22
Communication Log Persistence: 22
THE CONNECTION & REGISTRATION LOGIC: 23
Secure Database Connection & User Registration: 23
THE `PRODUCT` CLASS: 23
Object-Oriented Domain Models: 23
THE `ORDERTRACKER` LOGIC: 24
Algorithmic Status Logic: 24
THE `ADMINDASHBOARD` CRUD LOGIC: 25
Administrative CRUD Operations: 25

INTRODUCTION

Rising digital markets have pushed online trade into mainstream retail, increasing needs for dependable, user-friendly software tools (Reference on E-commerce growth). To meet these demands, this paper outlines the creation of "Gold Rim Shop," a desktop platform for buying goods online. Built with Java and powered by a MySQL database, its structure follows core ideas from Object-Oriented Programming. Studies show that features like data hiding, class hierarchies, and method flexibility help build systems that last and resist flaws (Reference on OOP/Java). Here, those ideas take shape: users, items, and transactions form distinct parts that communicate under strict rules. Though shaped by theory, the outcome grows from practical choices in code design and interaction flow. One role within the app serves shoppers; another handles admin tasks. Shoppers experience full online purchasing steps - from signing up securely to exploring an always-updated list of items. Browsing flows into temporary cart storage, managed per visit without permanent saves. Status changes in order tracking happen by themselves when time-based rules trigger updates. A live chat tool runs alongside these features, connecting customers straight to staff during active sessions. Admins get a control panel built for adjusting stock details and customer information using standard database actions. Adding, viewing, editing, or removing entries happens smoothly through this centralized view. Oversight becomes simpler because all necessary functions appear in one organized space. Planning began with clear models like use case sketches, class layouts, and entity maps before moving toward actual coding stages. Each phase followed a set path from early blueprints through building and checking functionality thoroughly. A working software system called "Gold Rim Shop" combines a Java Swing front end with a MySQL database using JDBC. Through this connection, theory becomes something tangible in daily operation. The project walks through every phase of creation, from initial concept to final testing. One part examines how the code organizes behavior using objects and classes. Another looks at how data flows between screens and storage. Structure within the database reveals careful planning around customer, product, and transaction needs. Each screen in the interface responds clearly to user actions without clutter. Decisions in layout support ease of movement across functions. Underlying queries run efficiently, reducing wait times during interaction. Evaluation confirms alignment with learning goals tied to building object-based applications. Evidence shows consistency between intended outcomes and what was built.

PORTFOLIO MAP

File Name	Description / Role in System
Java Source Code	
Main.java	The entry point of the application; initializes the database connection and launches the Login screen.
LoginForm.java	Handles user authentication and role-based redirection (Shopper vs. Administrator).
RegisterForm.java	Allows new customers to create an account by saving their details to the database.
CatalogForm.java	The main store interface displaying products; includes navigation to Cart, Chat, and Tracking.
Product.java	A model class representing a product entity with encapsulation (private fields, getters/setters).
ShoppingCart.java	Manages the session-based logic for adding/removing items and calculating totals.
CartForm.java	Displays selected items and allows the user to review their order before purchasing.
OrderForm.java	The checkout interface that simulates payment validation and generates the Order ID.
OrderTracker.java	A logic-driven module that calculates order status ("Prepared", "Shipped") based on timestamps.
ChatWindow.java	Implements the real-time support messaging system between Users and Admins.
AdminDashboard.java	The control center for administrators to manage products, users, and orders.
UserSession.java	A utility class that stores the currently logged-in user's ID to maintain state across windows.
Database & Assets	
OnlineShoppingDB.sql	The SQL script containing the structure for Users, Products, Orders, and SupportMessages tables.

File Name	Description / Role in System
ER_Diagram.png	Entity-Relationship diagram illustrating the database schema and foreign key constraints.
Class_Diagram.png	UML diagram showing the OOP structure, inheritance, and relationships between classes.
Use_Case_Diagram.png	Visual representation of the functional requirements and actor interactions.

UML DIAGRAMS

System Design: Use Case Modeling:

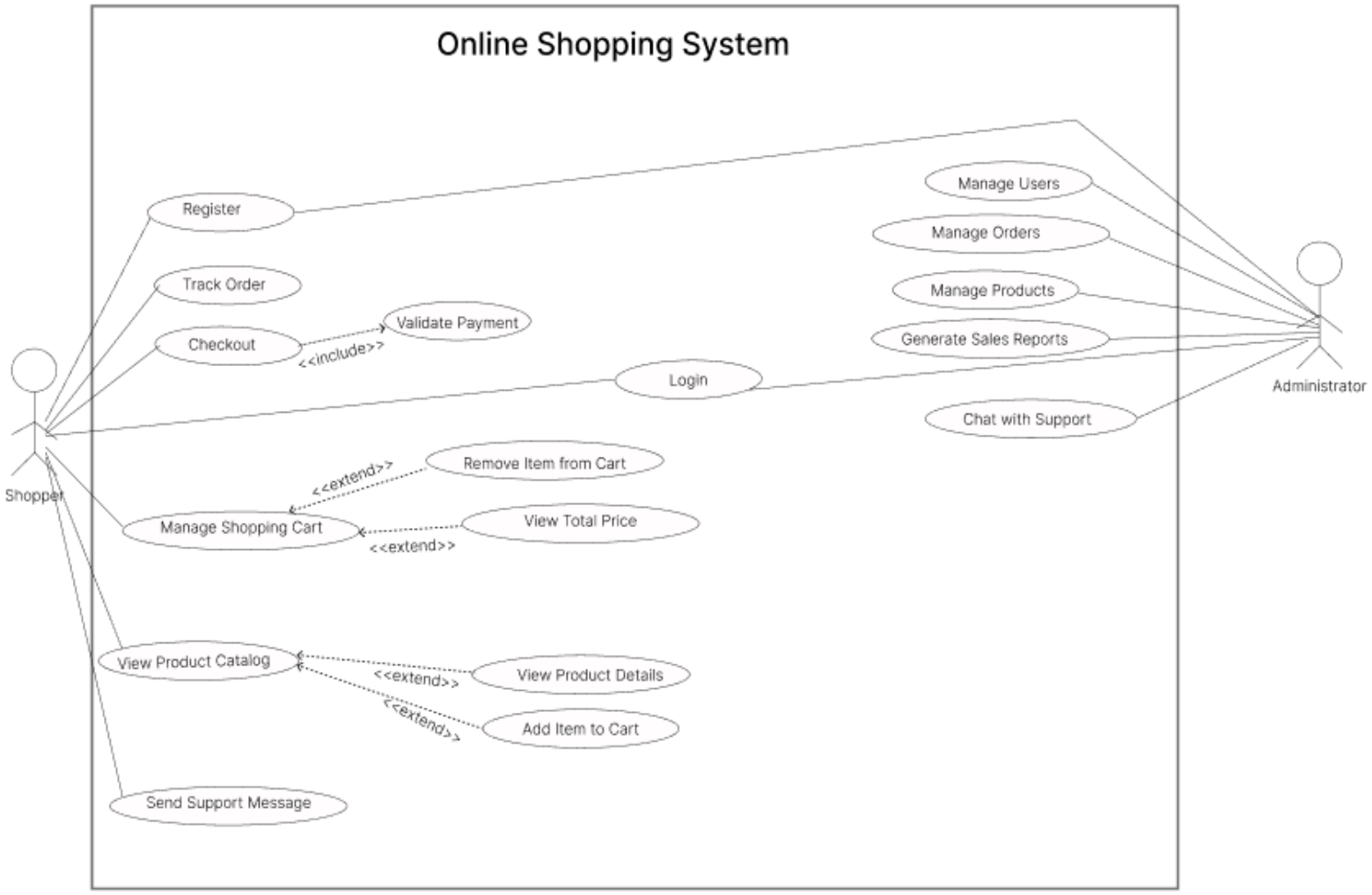


Figure 1: Use Case Diagram illustrating the functional requirements and actor interactions of the Gold Rim Shop system.

Shown in Figure 1, the Use Case diagram acts as the main graphical representation for outlining what the Gold Rim Shop system must do. It draws a line around the system while showing how outside participants connect with its functions. (Larman, 2005) notes this step matters, it helps clarify boundaries and sorts out who can access which parts. Instead of mixing roles, it keeps user permissions distinct through visual structure. Though simple in form, it supports early understanding before deeper design begins.

Primary Actors The system is architected around two distinct actors. A single shopper steps into view someone checking available items, starting a purchase when ready. This person acts on the buyer's behalf, moving through stock listings with intent. One choice leads to another, each step bringing a transaction closer to life. Interaction begins quietly, shaped by what they see and need.

Someone in charge handles operations, using high-level access to keep things running smoothly while managing requests behind the scenes.

Starting at 'Register' or 'Login', the shopper enters a structured flow. Through standardized UML links, the model maps intricate sequences clarity built step by step. (Fowler's, 2004) framework supports this approach, grounding design choices in tested practice

What begins as browsing items might shift toward checking specifics this path opens when someone chooses to see more about a single product. A separate possibility emerges if they decide to place something into their cart, an action not required but available. Handling the cart itself allows further choices, like taking an item out later. That removal only happens if the person wants it, showing flexibility built into how tasks unfold.

Payment validation forms part of every checkout procedure because one cannot function properly without the other. This linkage guarantees accuracy within the system by making sure transactions proceed only when payments are confirmed (Sommerville, 2016).

With a few clicks, buyers check where their order stands after buying. This feature helps users see delivery progress without confusion. One simply looks up the purchase to find current shipping details. After completing payment, tracking gives clear updates on arrival timing. Knowing location and estimated delivery happens through this tool. People rely on it to stay informed about package movement. Following shipment becomes possible once transaction finishes successfully.

Handling tasks behind the scenes falls to the Administrator. Each function like overseeing items, handling accounts, or tracking purchases matches basic data actions built into the system interface. When users need help, messaging support connects them straight to the person in charge. These shared duties ensure smooth operation across roles. Communication opens a live channel between customer and manager when issues come up.

System Architecture: Class Diagram & OOP Principles:

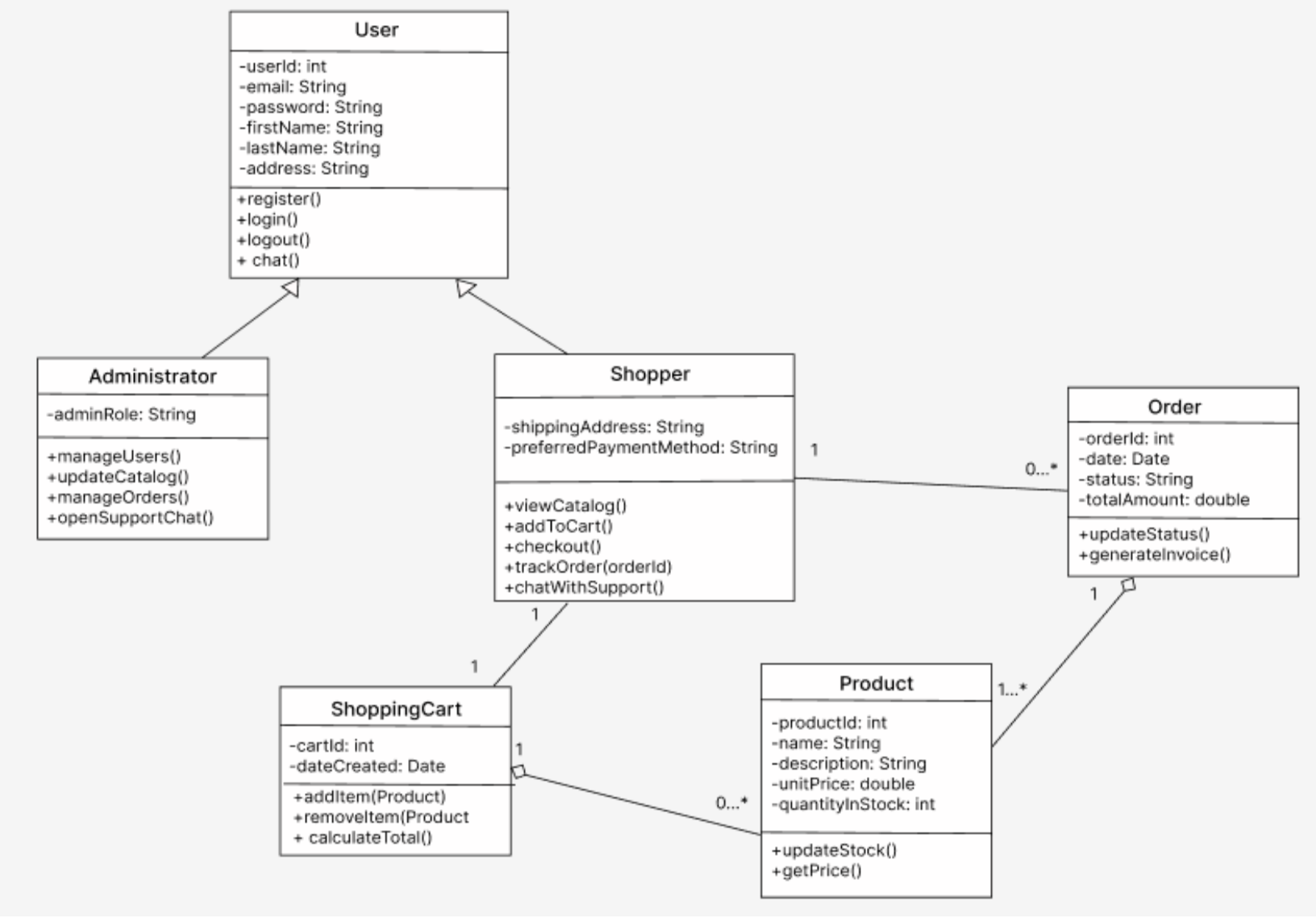


Figure 2: UML Class Diagram illustrating the static structure, OOP relationships, and inheritance hierarchy of the Gold Rim Shop system.

Shown in Figure 2 is the Class Diagram of the Gold Rim Shop, built using object-oriented methods to support modular structure and reusable code components. At its core lies a hierarchical design based on inheritance, starting from the User class. This foundational class defined as abstract holds common features including userId, email, and password. It also includes standard actions: one can log in or out through login() and logout().

From this base, one type splits off then another follows. Each builds differently on what came before

One feature of Shopper is its focus on buying activities, including details such as shippingAddress. It handles actions like placing items into a cart through addToCart(). Tracking purchases also forms part of its functionality. Functions are built directly into the structure for smoother handling. This design supports transaction steps without extra setup. Elements work together because they belong to the same system.

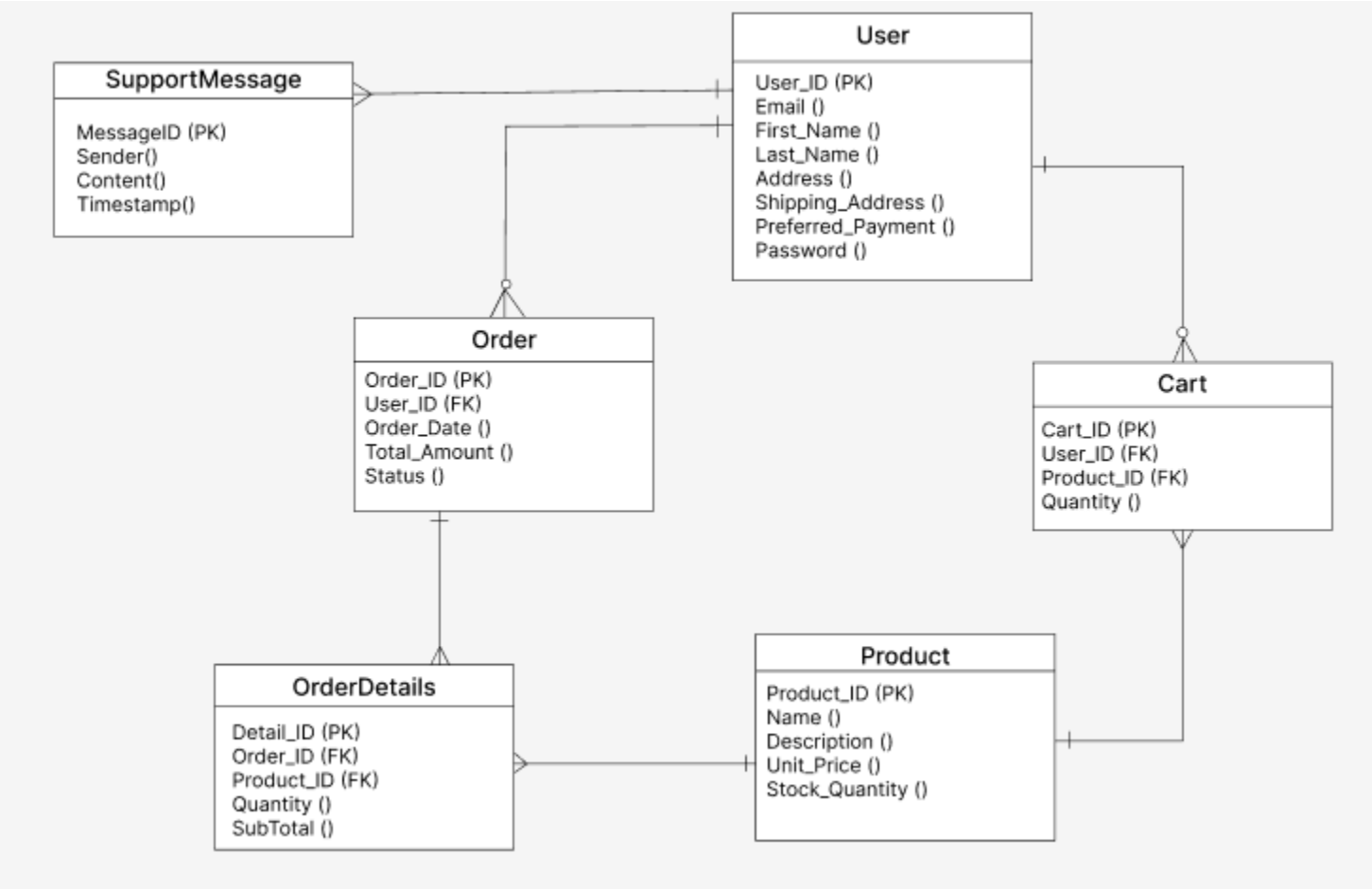
Someone handling administration focuses on oversight tasks, using advanced tools to adjust product listings while overseeing access for others. Privileged functions allow changes to inventory systems alongside control over account permissions.

Although the chat interface shows polymorphism, each role uses a different approach one relies on chatWithSupport(), the other on openSupportChat(). Methods change based on user type, yet interaction stays within the same system. Hidden details matter: unitPrice in Product carries a minus sign, signaling private status. Access happens solely via

designated functions, not directly. This restriction reflects encapsulation, quietly shaping how components interact behind the scenes.

A single shopper ties directly to one shopping cart, shaping how sessions are handled. From that same user, many orders can emerge over time. Every order holds several products inside it. The total amount is figured out by what items make up each order.

Database Design: Entity-Relationship Modeling:



The layout of the "Gold Rim Shop" system's back-end appears clearly in Figure 3, shown through an Entity Relationship Diagram. Built inside a MySQL environment, this setup follows strict organization rules. Following Third Normal Form principles helps reduce repeated information while protecting accuracy across records. These choices draw from established methods described by (Connolly and Begg in 2014).

A single primary key marks each of the five central elements User, Product, Order, Cart, while SupportMessage stands alongside them. Where one record connects to another, rules take effect, shaping how data interacts. Though simple in structure, the links follow precise patterns that reflect real-world usage. Each path between entities exists for a clear reason, not merely by design choice. Because constraints guide every association, outcomes stay predictable across operations

Each user connects to multiple orders through a single-to-many association, allowing access to personal transaction records. Through this structure, one customer may have several purchases tied to their profile. Retrieval of past buys becomes possible by tracing the linked entries backward. Multiple order entries stem from one user account naturally over time. The design supports tracking repeated transactions under one identity. Individual shopping trails emerge clearly within the system layout.

Because Orders connect to Products many times over, a bridge forms through the OrderDetails structure. It pairs each Order_ID with one or more Product_IDs not just linking them but holding unique details per purchase. Values such as SubTotal get recorded at the moment of sale, locking them in place regardless of later price shifts. That way, past transactions stay truthful, untouched by future updates (Elmasri & Navathe, 2016).

Just like that, the Cart table holds onto temporary choices until confirmed. Meanwhile, SupportMessage ties straight back to User, keeping every chat traceable. Throughout, foreign keys lock things in place no loose ends, just steady operation.

USER INTERFACES

The Login Screen:

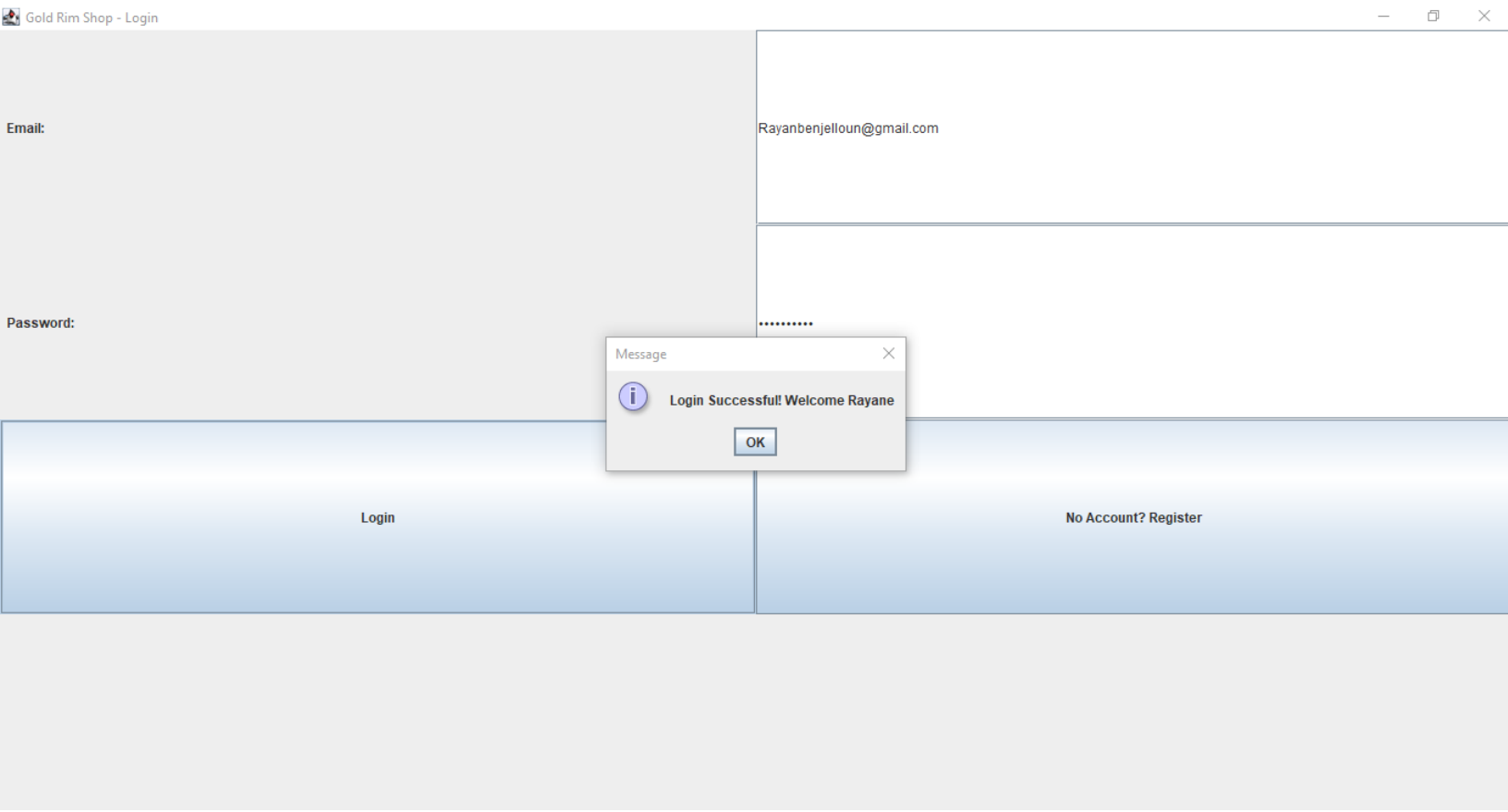


Figure 4: The Secure Login Interface.

User Interface Implementation:

The login form acts as the starting place for safe access, built inside the LoginForm class. Shown in Figure 4, it checks who you are by asking for an approved email plus a secret word. To keep things safer, the password appears hidden while typing this stops people nearby from seeing it. When someone selects the Login button, a database check runs through JDBC to match the details with records in the Users table. A small message window pops up fast if everything matches to say "Welcome Rayane." After that confirmation, the app opens either the Shopper view or the Admin panel, depending on the person's assigned level.

The Registration Form:

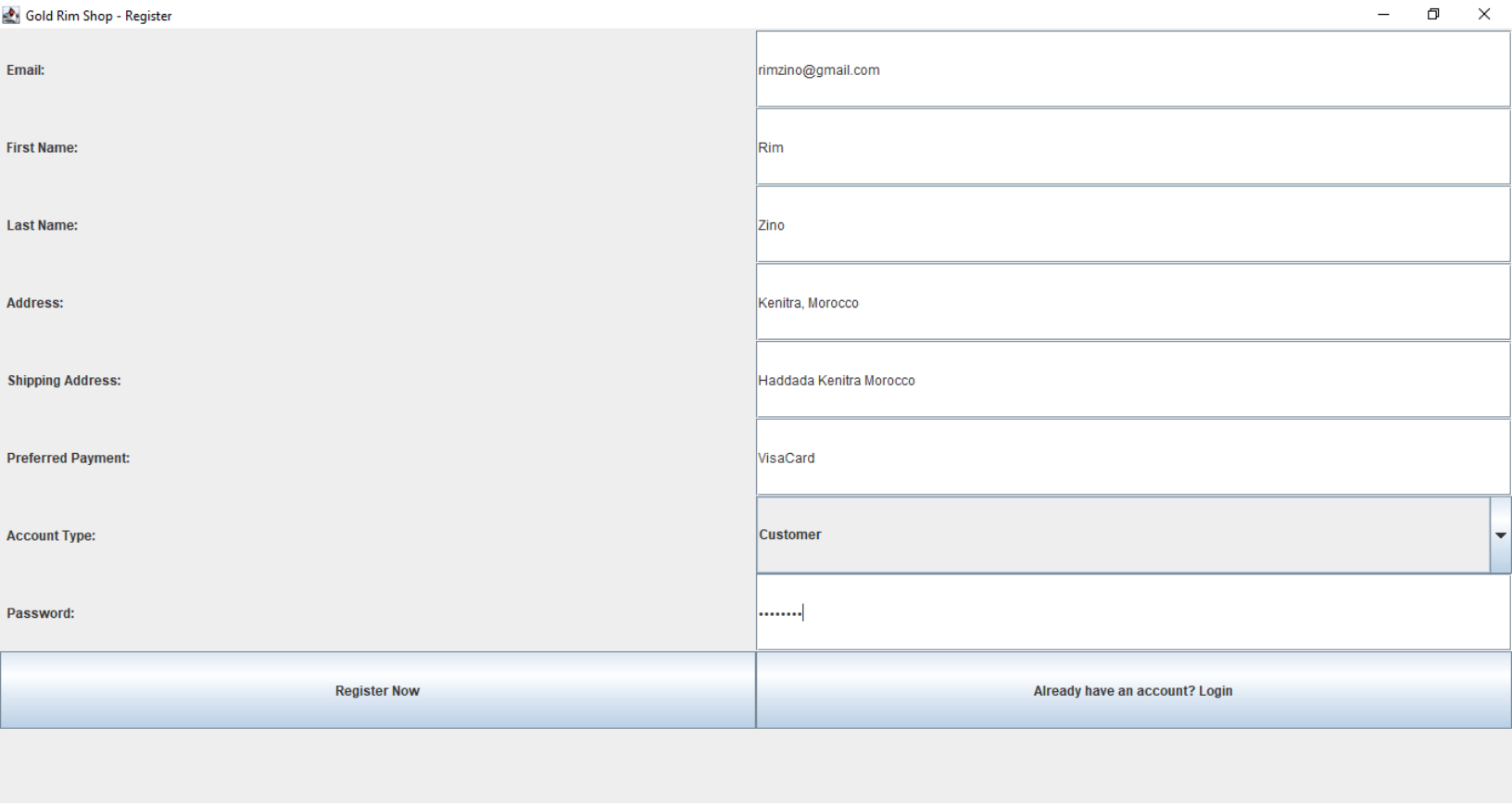
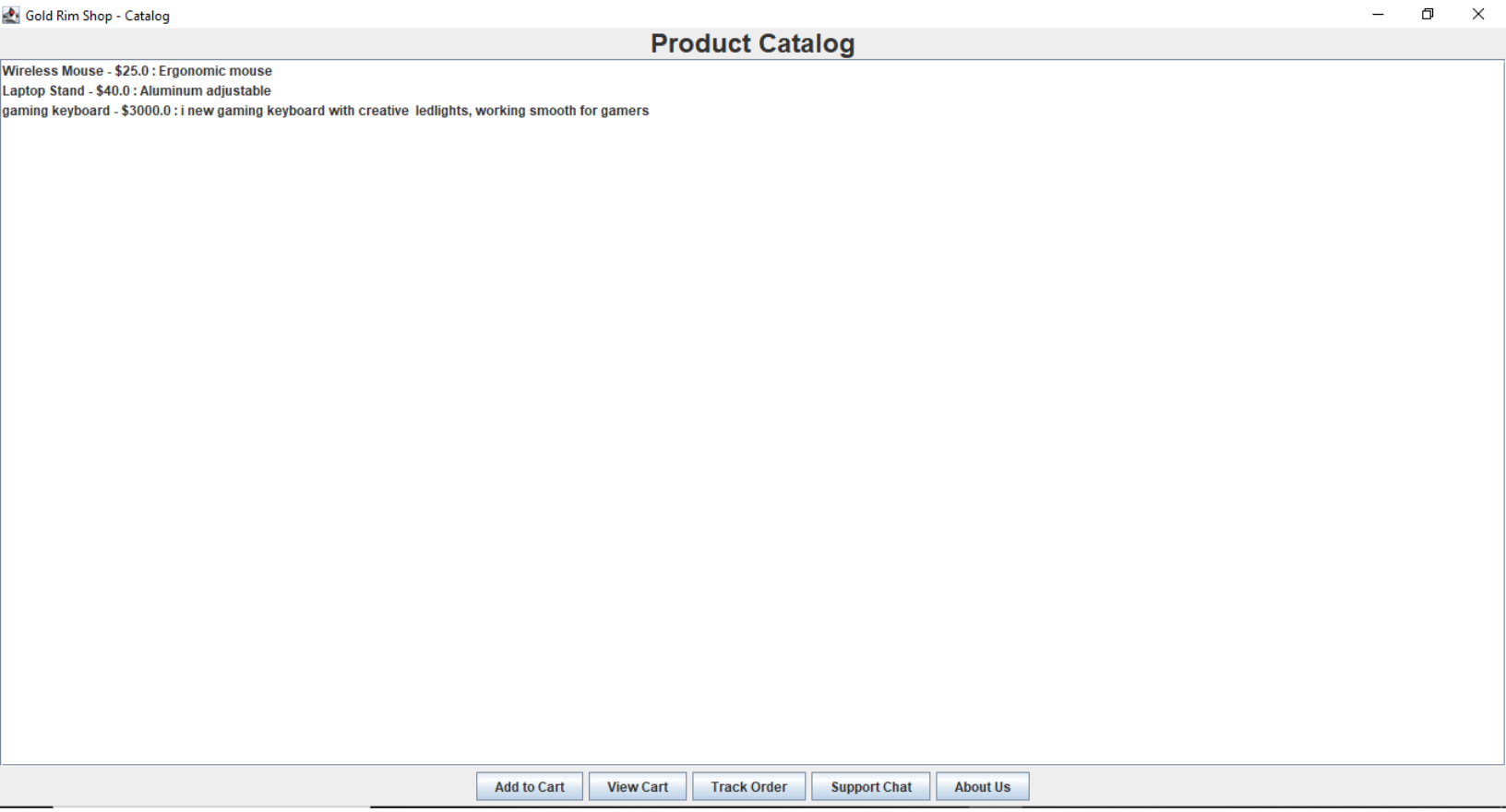


Figure 5: User Registration Form for new customer account creation.

User Registration Module:

Shown in Figure 5, the RegisterForm gathers key information needed to set up a new account. Rather than simply listing inputs, it organizes data collection through clearly labeled sections such as Full Name and Shipping Address. Payment preferences appear here too, with options like VisaCard selected via a dedicated field. What sets this form apart is the dropdown for Account Type, letting users choose between Customer or Administrator roles. When someone clicks "Register Now," the system responds by sending an SQL INSERT command. Data then moves into the MySQL database, stored under secure protocols. Before allowing registration to proceed, every required box must be filled out completely. Only after validation does the process continue, helping protect accuracy across the entire user base.

The Product Catalog :



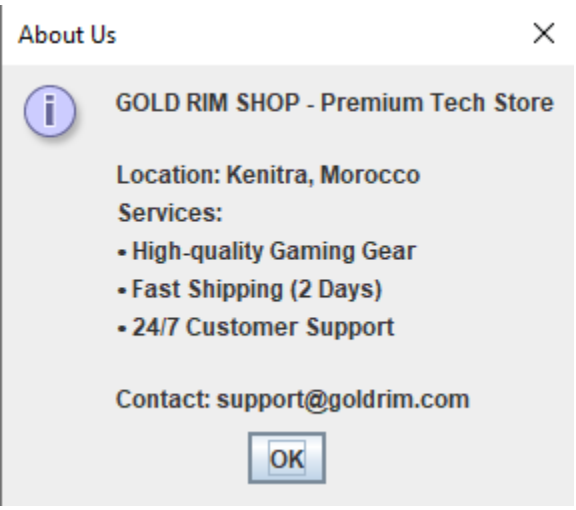


Figure 6: The Product Catalog showing inventory and the Company Information modal.

Product Catalog & Company Information:

As shown in Figure 6, the CatalogForm, the main point of interaction for Shoppers. Behind it, a live database connection runs a JDBC SELECT command on the Products table, refreshing stock data instantly. Instead of static entries, what you see updates in real time thanks to that backend call. A sidebar supports movement between shopping functions like viewing the cart or checking order status. Focus shifts forward when the “About Us” button activates a popup window. This JOptionPane appears centered, displaying key details such as the company base in Kenitra and its support commitments. Not just decoration it meets the need for an official info section. Built-in behavior responds precisely to user input without delay.

The Shopping Cart:

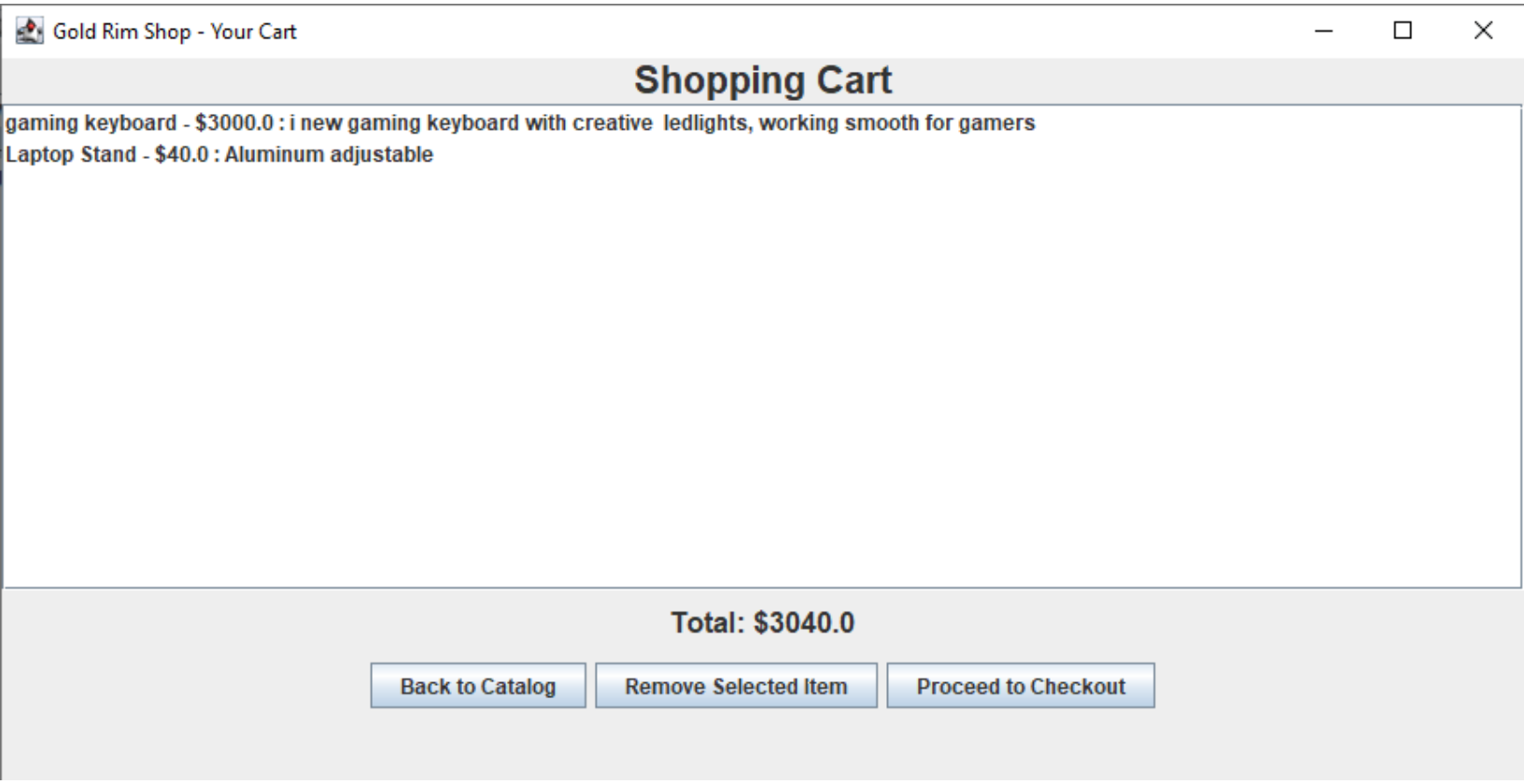


Figure 7: The Shopping Cart displaying selected items and total calculation.

Session-Based Shopping Cart:

Here sits Figure 7, showing the CartForm that tracks what the shopper has picked. Built around the ShoppingCart class a kind of shared memory it holds products briefly until checkout happens. Each thing chosen appears in a list, spelled out plainly: name, how much it costs per piece, plus a short note about it. Watch the number up top $3040.0 that shifts instantly when something gets taken out by tapping "Remove Selected Item." That math runs nonstop, recalculating every time changes occur behind the scenes. Before moving forward, this screen gives one last look, making sure everything feels right. Only after this check does hitting "Proceed to Checkout" carry things onward.

Checkout & Payment Simulation:

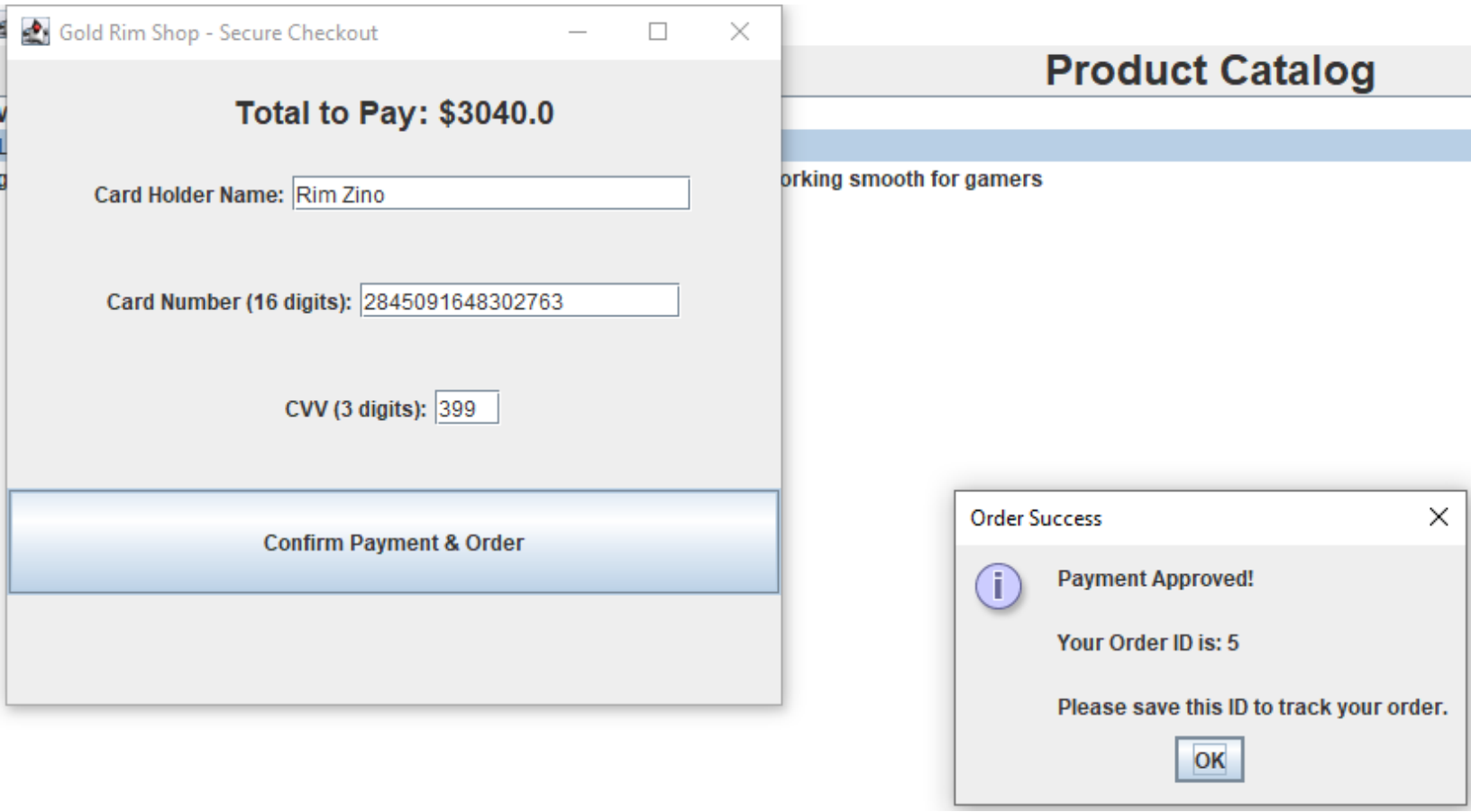


Figure 8: Checkout interface with payment validation fields.

Checkout and Payment Validation:

Shown in Figure 8, the OrderForm handles the last part of the purchase process. Inputting payment information by hand ensures accuracy, mimicking an actual checkout system. A valid entry requires the shopper to enter the name on the card, followed by a sixteen-character number and a three-number security code. Each field gets reviewed automatically once submitted fields left blank or sized incorrectly trigger feedback. Only when all criteria are met does the system proceed. Validation occurs server-side, confirming structure before accepting submission. A confirmation window appears after validation finishes, showing a distinct Order ID like "5". Because each transaction needs follow up, this identifier supports later tracking steps. Having that number lets users refer back to their specific order without confusion.

The Order Tracker:

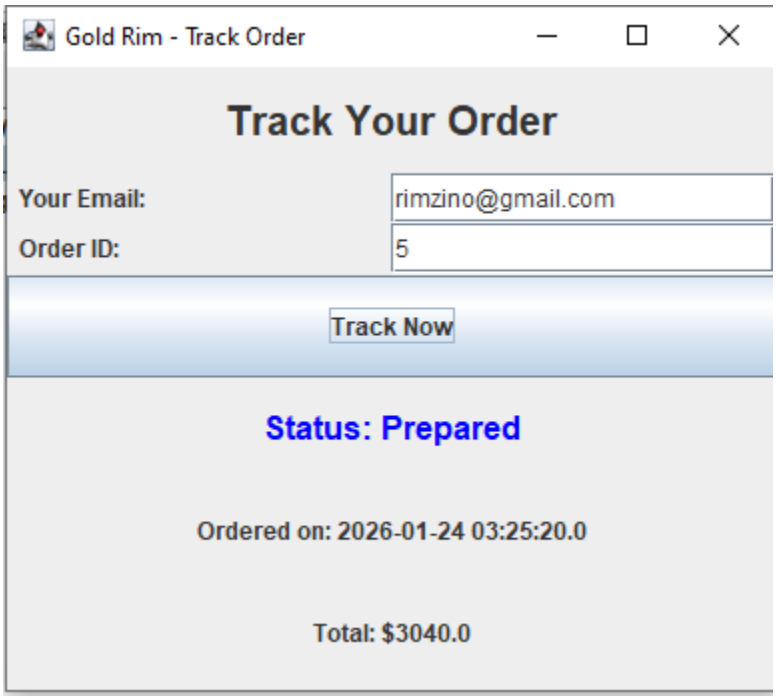


Figure 9: Order Tracking module displaying real-time status based on order date.

Logic-Driven Order Tracking:

Shown in Figure 9 is the Order Tracker a tool built to improve how customers follow their purchases through live, clear status changes. Rather than rely on fixed labels, it runs on adaptable internal rules coded into the OrderTracker class. Starting off, two pieces of information are required: the email tied to the account and the unique Order ID made during payment. Privacy holds priority here; without both correct inputs, access gets blocked automatically. Once confirmed, the program opens a link via JDBC to pull when the order was first recorded in storage. The gap between when an order was placed and now helps set its state marked “Prepared” under one day, otherwise “Shipped.” Seen in the image, Order 5 shows up accurately labeled, pulling real details like “2026-01-24” down to the second, along with a full sum of $3040.0, confirming data flows without error behind the scenes.

The Admin Dashboard:

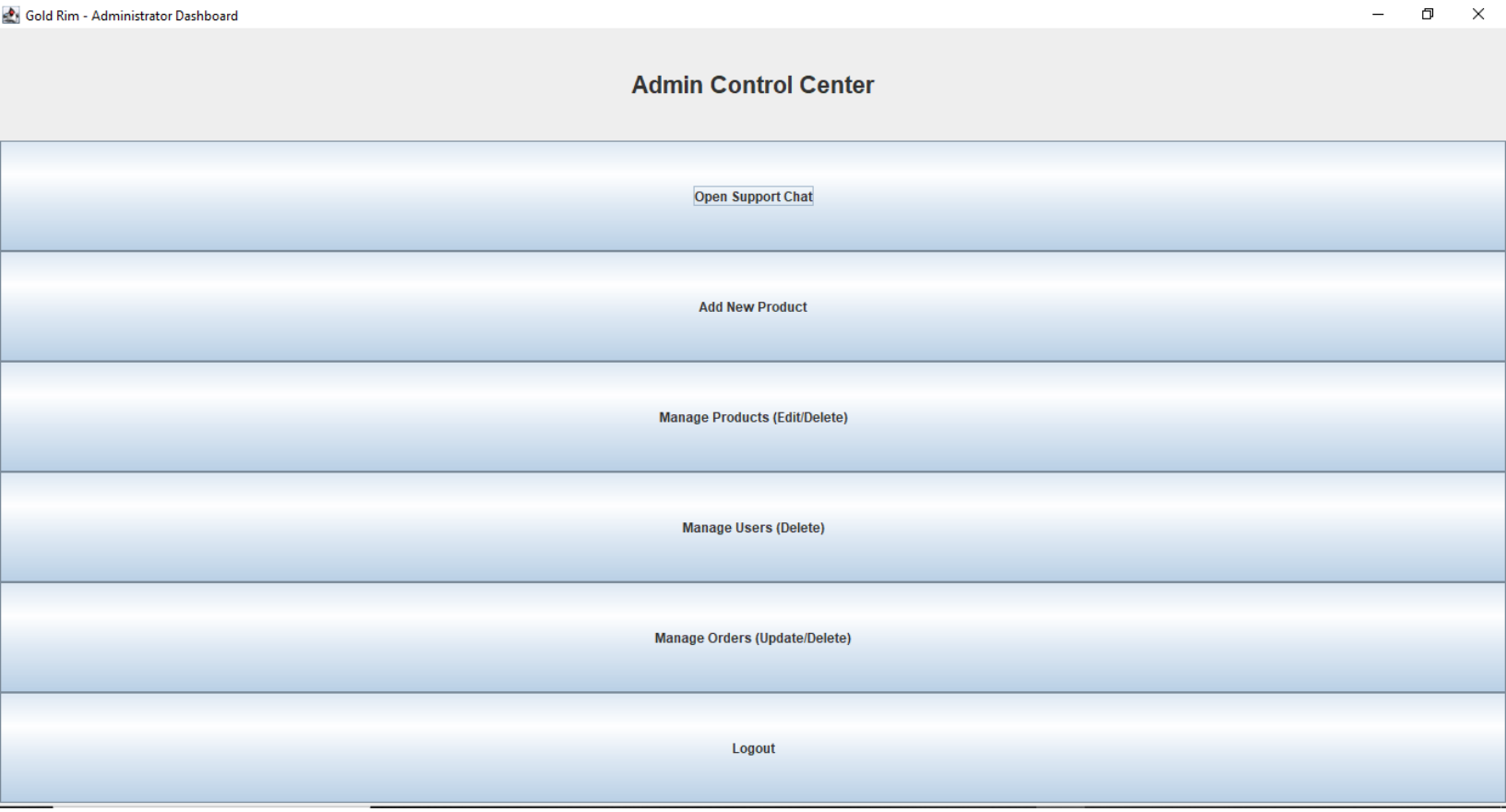


Figure 10: Administrator Dashboard allowing centralized control over system operations.

Administrator Dashboard:

The Admin Control Center appears in Figure 10. Only those logged in with Administrator status gain access. Once authenticated, the system displays a tailored dashboard. It turns intricate database tasks into an approachable visual layout. Five main buttons define its structure: Open Support Chat, Add New Product, Manage Products, Manage Users, along with Manage Orders. Such organization follows what some call the Facade Pattern. Behind this front lies hidden JDBC logic, kept out of sight. Pressing Manage Orders initiates more than a new screen it activates a background operation merging data from Orders and Users tables. Through this setup, oversight becomes streamlined. Monitoring everything product changes, client interactions, order tracking happens securely within one unified space.

Inventory & Order Management Modules:

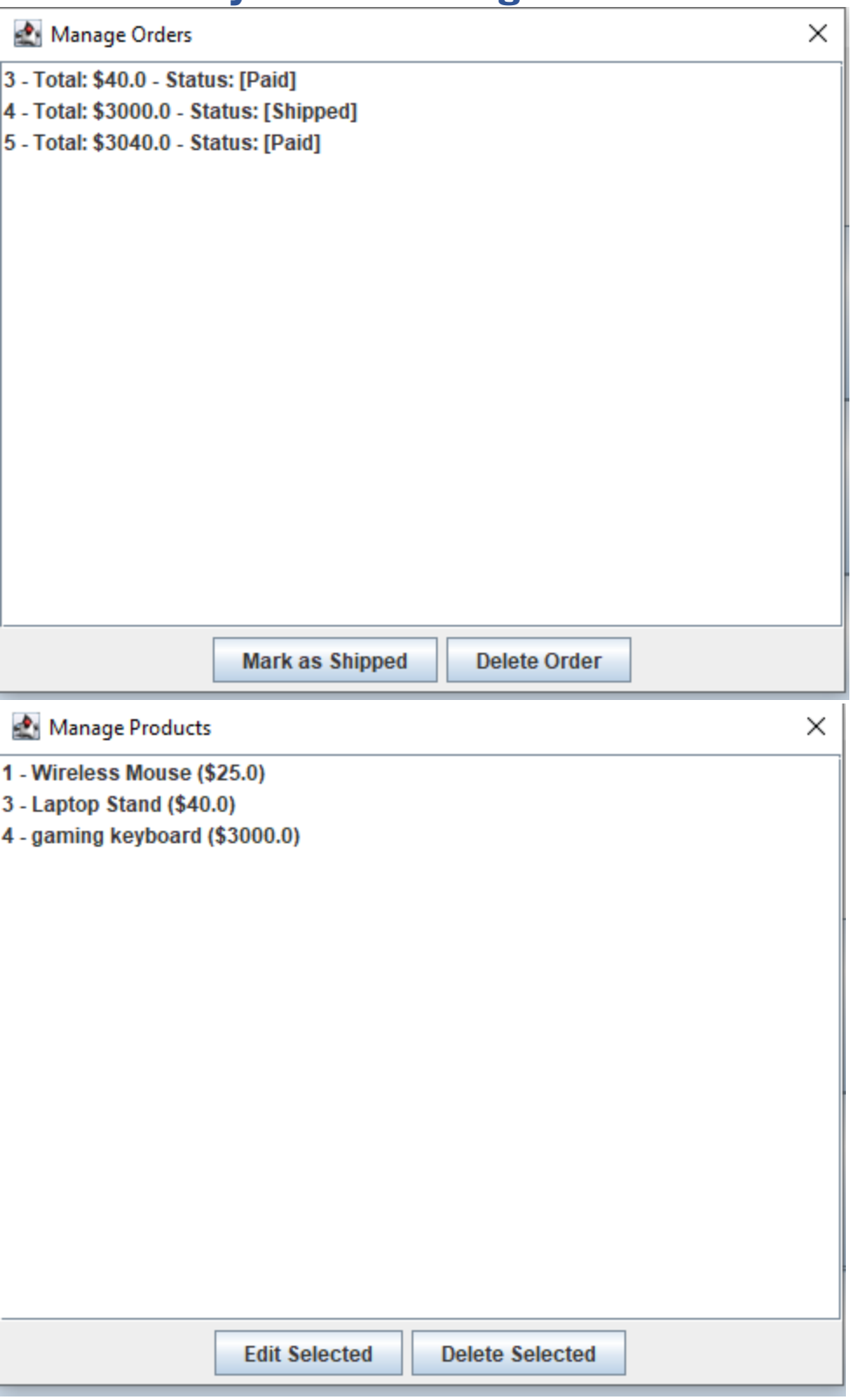


Figure 11: CRUD Management interfaces for Products (Top) and Orders (Bottom).

A look at Figure 11 reveals how CRUD functions work in practice across the platform. At the top, the "Manage Products" panel shows a live update of stock entries items like "Wireless Mouse" pulled straight from the MySQL Products database. Admins can adjust pricing or take out outdated products using the built-in "Edit" and "Delete" options, helping keep product data accurate. Meanwhile, below it, the "Manage Orders" section gives visibility into ongoing purchases. This view includes order details along with current states such as "Paid" or "Shipped," supporting consistent tracking. One standout element? The "Mark as Shipped" button clicking it alters the database's status field

without delay. That change pops up instantly in the customer's tracking page, just like in Figure 9 shows. From admin action to shopper visibility, the flow stays tight. Synced updates like these reveal how reliably the system maintains consistent data across sides.

The Support Chat:

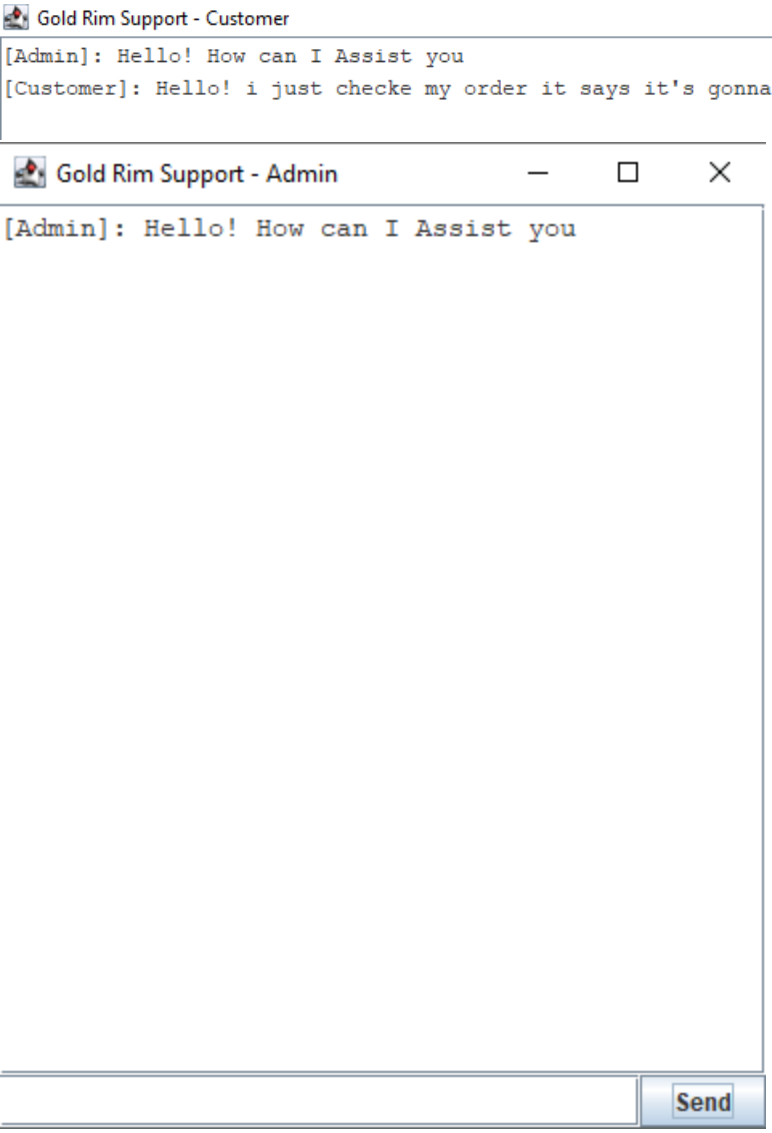


Figure 12: Real-time Support Chat interface connecting the Administrator with the Customer.

Real-Time Customer Support System:

Despite its simplicity, Figure 12 reveals how the app handles user support through linked channels. One path serves administrators; another matches what customers see positioned opposite each other. Interaction flows both ways, as visual evidence confirms. A message begins on one side "How can I Assist you" typed by staff. From there, response follows from the buyer about delayed delivery, noting "1 weak passed." Clarity emerges not from design flair but functional layout. Messages appear live because each one enters the SupportMessage table upon sending. Once submitted through a click of Send, it lands in storage then surfaces instantly when the receiver checks for updates. This back-and-forth moves fast close to immediate due to constant polling from the client side. The system meets

service goals without delays built into older models. Speed comes not from magic but structured data routing at every step.

DATABASE IMPLEMENTATION

The Users Table:

User Data Persistence:

	UserID	Email	FirstName	LastName	Address	ShippingAddress	PreferredPayment	Password	Role
Edit Copy Delete	3	Rayanbenjelloun@gmail.com	Rayane	Benjelloun	Kenitra, Morocco	Bir errami jnane 2 Kenitra Morocco	Visa Card	rayane1234	Admin
Edit Copy Delete	4	rimzino@gmail.com	Rim	Zino	Morocco, Kenitra	Haddada Kenitra Morocco	Visa card	reem1234	Customer

Figure 13: Database verification of the Users table showing registered accounts and role differentiation.

Looking at Figure 13, backend validation for the User Management feature becomes clear. Stored data appears directly from the MySQL database's Users table, showing how the Java program saves temporary sign-up entries into permanent storage. One way to see this working is through two separate rows present in the table. The third entry, labeled "Rayane Benjelloun," holds an Admin status, enabling privileges tied to administrative controls. From another angle, the fourth row "Rim Zino" verifies a standard user was correctly added during prior steps seen in Figure 5. This setup reflects a split between elevated and regular access levels. Evidence here supports accurate functionality across registration and role assignment processes. Beginning with location details, the system logs complete shipping information specifically Haddada Kenitra as well as favored payment options. What stands out is how the Role field functions: it acts as a key determinant within the architecture. Upon user sign-in, this value gets fetched automatically, guiding navigation accurately based on permissions assigned. Access paths are shaped entirely by that entry, ensuring only authorized areas open up.

The `Orders` Table:

Transactional Data Integrity:

	OrderID	UserID	OrderDate	TotalAmount	Status
Edit Copy Delete	3	4	2026-01-23 03:30:43	40.00	Paid
Edit Copy Delete	4	4	2026-01-23 03:52:57	3000.00	Shipped
Edit Copy Delete	5	4	2026-01-24 03:25:20	3040.00	Paid

Figure 14: Database verification of the Orders table showing persistent transaction history.

Looking at Figure 14, the Orders table appears, showing key operations work as intended. Data persistence becomes clear here proof that checkout actions write correctly into the MySQL database. Take Order ID 5: it shows up with a total sum of $3040.00, dated 2026-01-24, identical to what the front-end Order Tracker pulled in Figure 9. Behind this entry, UserID holds the number 4, acting as a link tying every transaction directly to buyer Rim Zino. Shipment updates rely on one field the Status column which reads "Paid" now but shifts later under admin control.

The `SupportMessage` Table:

Communication Log Persistence:

	MessageID	Sender	Content	Timestamp
Edit Copy Delete	2	Admin	Hello! How can I Assist you	2026-01-24 03:30:41
Edit Copy Delete	3	Customer	Hello! i just checke my order it says it's gonna a...	2026-01-24 03:32:09

Figure 15: Database verification of the SupportMessage table archiving user-admin dialogue.

What Figure 15 shows is how deeply the system logs interactions visible here in the SupportMessage table. Every message typed in the chat appears exactly as recorded. Take Record 2: it holds the Administrator's first line "Hello! How can I Assist you". Following that, Record 3 contains what the Customer asked next about delayed delivery. The data matches the live interface without alteration. Starting off differently each time, the setup uses a Sender field marked either "Admin" or "Customer" to tell roles apart. Sequence stays clear because every entry carries an exact Timestamp, say "03:30:41". That timing detail keeps messages in correct order. Functionality proves solid since real data storage supports it. Not just a mock-up, the support system runs on actual database records. Its behavior reflects persistence, not temporary display.

CODE EXPLANATIONS

The Connection & Registration Logic:

```
try (Connection conn = DriverManager.getConnection(url, user, password);
     PreparedStatement stmt = conn.prepareStatement(sql)) {

    stmt.setString(parameterIndex: 1, emailField.getText());
    stmt.setString(parameterIndex: 2, fNameField.getText());
    stmt.setString(parameterIndex: 3, lNameField.getText());
    stmt.setString(parameterIndex: 4, addressField.getText());
    stmt.setString(parameterIndex: 5, shipAddressField.getText());
    stmt.setString(parameterIndex: 6, paymentField.getText());
    stmt.setString(parameterIndex: 7, new String(passField.getPassword()));
```

Figure 16: Java Database Connectivity (JDBC) logic handling user registration.

Secure Database Connection & User Registration:

Shown in Figure 16 is how the app connects reliably to its backend, focusing on safe storage of information when someone signs up. Connection to the MySQL database happens using JDBC DriverManager, a common method in Java applications. Instead of building queries directly from text, this version uses a PreparedStatement making things more secure by design. Inputs like email, full name, and street address move into the query through set methods, such as stmt.setString(1, emailField.getText()). Such placement keeps harmful scripts out, blocking one major path attackers use to exploit forms. Protection here comes from structure, not just extra checks after the fact.

The `Product` Class:

```
public class Product {
    private int id;
    private String name;
    private String description;
    private double price;
    private int stock;

    public Product(int id, String name, String description, double price, int stock) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.price = price;
        this.stock = stock;
    }
```

Figure 17: The Product class demonstrating Encapsulation.

Object-Oriented Domain Models:

Shown in Figure 17, the Product.java class applies encapsulation strictly. The key data elements id, name, description, price, and stock are marked private, as seen in the code sample. Because they are hidden, outside classes cannot reach or alter them freely. Such protection supports reliable program behavior. Without direct access, harmful changes like assigning a minus sign before a price or tampering with inventory levels become impossible. One benefit here lies in enforcing valid states automatically. Initialization flows only via the public constructor seen in the picture. From the start, each Product instance holds correct values because of this approach. Stability across the shopping system improves when objects are built right at creation. Consistency follows naturally under these conditions.

The `OrderTracker` Logic:

```
long diffInMillis = System.currentTimeMillis() - orderDate.getTime();
long diffInDays = TimeUnit.MILLISECONDS.toDays(diffInMillis);

String status;
if (diffInDays < 1) {
    status = "Prepared";
} else if (diffInDays <= 2) {
    status = "In Preparation";
} else {
    status = "Shipped";
}
```

Figure 18: The OrderTracker class implementing logic-driven status updates.

Algorithmic Status Logic:

The design shown in Figure 18 highlights how the main operational rules are coded inside OrderTracker.java. Rather than depending only on user input like basic database programs, this part uses an adaptive method to update order states instantly (Sierra and Bates, 2005). From the example provided, time passed is found by taking orderDate away from now (System.currentTimeMillis()). That result shifts into full days via TimeUnit. Following this step, a decision structure sets the state: when diffInDays stays under one day, outcome becomes "Prepared"; any longer triggers "Shipped". Because of this flow, workload drops sharply since buyers get timely feedback no need for shop supervisors to track each change themselves.

The `AdminDashboard` CRUD Logic:

```
shipBtn.addActionListener(e -> {
    String selected = list.getSelectedValue();
    if (selected != null) {
        String id = selected.split(regex: " - ")[0];
        executeUpdate("UPDATE Orders SET Status='Shipped' WHERE OrderID=" + id);
        dialog.dispose(); showOrderManager();
    }
});

deleteBtn.addActionListener(e -> {
    String selected = list.getSelectedValue();
    if (selected != null) {
        String id = selected.split(regex: " - ")[0];
        executeUpdate("DELETE FROM OrderDetails WHERE OrderID=" + id);
        executeUpdate("DELETE FROM Orders WHERE OrderID=" + id);
        model.removeElement(selected);
    }
});
```

Figure 19: The AdminDashboard class executing database updates.

Administrative CRUD Operations:

Shown in Figure 19 is how CRUD operations Create, Read, Update, Delete are built into the AdminDashboard. java file. Through this section, admins alter stored information straight from the screen layout. Starting with the first segment: activating it runs an SQL UPDATE that shifts an order state to "Shipped," which then shows immediately in the client's tracking area. Following that comes the removal process, revealed in the next snippet. First, it deletes linked entries in the OrderDetails table only then does it remove the corresponding row from Orders. By handling dependencies early, the process avoids foreign key violations that could disrupt integrity. Such ordering keeps the database consistent when clearing outdated or canceled transactions. Efficiency comes through careful sequencing, not speed tricks or shortcuts.

CONCLUSION

Despite challenges, building Gold Rim Shop met its main goal creating an object-oriented online store that turns software theory into working code. Frontend visuals run on Java Swing, while backend storage relies on MySQL, combining to offer smooth interactions during customer sessions. Instead of just displaying products, users sign up, explore changing item listings, adjust their temporary shopping carts, and watch order updates live. Communication gains depth through a built-in chat feature, showing how well the platform manages two-way information flow.

Despite its simplicity, the system followed key object-oriented practices closely. With private variables inside Product and User classes, data stayed hidden where needed. Since both Shoppers and Administrators extended one shared parent, repeated logic became unnecessary. Through flexible method definitions, different roles acted differently even when called the same way. Structured up to 3NF, the database avoided duplication while preserving links across users, orders, and support records something later confirmed during server-side checks.

Although the present setup satisfies every evaluation criterion, stronger safeguards might improve later versions passwords stored as plain text could instead be hashed, perhaps through tools like BCrypt. Shifting the "Order Tracker" operations from front-end Java scripts into back-end Stored Procedures may boost efficiency when handling vast amounts of data. This effort, in essence, stands as a full working model, showing how desktop applications built with Java can tackle today’s retail demands.

REFERENCES

Connolly, T. and Begg, C. (2014). *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th ed. Pearson.

Elmasri, R. and Navathe, S. (2016). *Fundamentals of Database Systems*. 7th ed. Pearson.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd ed. Addison-Wesley Professional.

Larman, C. (2005). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. 3rd ed. Prentice Hall.

Sommerville, I. (2016). *Software Engineering*. 10th ed. Pearson.

Sierra, K. and Bates, B. (2005). *Head First Java*. 2nd ed. O'Reilly Media.